

Does it matter whether misfitting items share a dimension?

Attenuation of item fit as a function of the correlation between misfit sources

Magnus Johansson

2026-08-01

Table of contents

Question	1
The structural constraint	1
Design	1
Attenuation of the on-target item (q5)	4
Detection follow-up	5
Detection of q5, across the correlation sweep	7
False positives on the seven fitting items	8
The $c = 0$ anchor	9
Notes for the write-up	10
References	10

Question

When two items misfit, does each one's apparent misfit depend on **whether they misfit for the same reason**? Specifically: two items loading on a single second dimension, versus one item on a second dimension and one on a third.

Informal earlier work suggested little or no difference. This checks that, and identifies the parameter that actually governs it.

The practical stake is the one-at-a-time removal advice in Johansson (2025): if a second misfitting item attenuates the first item's fit statistic, each departure is being measured against a reference (the conditioning score and the CML parameters) that has already partly absorbed it.

The structural constraint

This turns out to be the key point. Let the main dimension be `theta1` and let the two misfitting items load on `theta2` and `theta3`, each correlated `rho` with `theta1`. Write `c = cor(theta2, theta3)`.

The correlation matrix

```
[ 1  rho  rho ]
[ rho 1  c  ]
[ rho c  1  ]
```

has determinant $1 + 2\rho^2c - 2\rho^2 - c^2$, which is non-negative only for

$$c \geq 2\rho^2 - 1$$

So **two “separate” dimensions cannot be made independent** when both load substantially on the main dimension. At `rho = 0.85` the minimum possible correlation between them is **0.445**, and under conditional independence given `theta1` it is `rho^2 = 0.72`. Only when the misfit is strong (low `rho`) does genuine separation become available.

That predicts the informal finding directly: at realistic misfit levels, “different dimensions” is nearly the same condition as “same dimension”.

Design

Rather than three discrete conditions, `c` is swept across its whole admissible range, so the gradient is measured rather than inferred. The sweep is parameterised as a fraction `t` of the available range:

$$c = c_{\min} + t(1 - c_{\min}), \quad c_{\min} = \max(0, 2\rho^2 - 1)$$

`t = 1` is always “same dimension” (`c = 1`); `t = 0` is as separate as the geometry permits. This keeps conditions comparable across `rho`.

- Arm: **phq9** – 9 items, CML parameters from the packaged real responses, latent SD estimated from the same data
- Misfitting items: **q5** (-0.15, on target) and **q8** (+1.40, off target), as in the other studies in this series

- ρ in {0.85 (calibrated to real phq9 misfit), 0.50}
- t in {0, 0.25, 0.5, 0.75, 1}
- **Baseline:** only q5 misfits ($k = 1$), the reference against which attenuation is measured
- **Noise anchor:** both items misfit through independent random responding — a genuine $c = 0$ condition reachable at any ρ , calibrated to match the $\rho = 0.85$ baseline infit
- n in {250, 1000}; 300 replications

Only item fit is computed — no bootstrap — so this runs in minutes. Detection consequences would need cutoffs and are left as a follow-up.

```
suppressMessages({
  pkgload::load_all("../", quiet = TRUE)
  library(MASS)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})

RHOS    <- c(0.85, 0.50)
TS      <- c(0, 0.25, 0.5, 0.75, 1)
NS      <- c(250, 1000)
R_REPS  <- 300
NOISE_P <- 0.15          # matches rho = 0.85 baseline infit (see mechanism study)
NCORES  <- 10
RES     <- "infit_misfit_source_correlation.rds"
```

```
load("../data/phq9.rda")
PH_OBS <- as.matrix(phq9[, paste0("q", 1:9)])
PH_OBS <- PH_OBS[complete.cases(PH_OBS), ]
PH_THR <- .fit_cml_thresholds(PH_OBS)
PH_LOCS <- vapply(PH_THR, mean, numeric(1))
.g <- seq(-6, 6, length.out = 81)
PH_SD <- .estimate_prior_sd(
  .grid_loglik(PH_OBS, .logp_tables(PH_THR, .g), .g), .g, 0)
ITEM_A <- 5L # on target
ITEM_B <- 8L # off target
MAXCAT <- 3L

c_min <- function(rho) max(0, 2 * rho^2 - 1)
c_of  <- function(rho, t) c_min(rho) + t * (1 - c_min(rho))

# admissible range and the conditional-independence value, per rho
data.frame(rho = RHOS) |>
  mutate(c_min = round(sapply(rho, c_min), 3),
         c_cond_indep = round(rho^2, 3),
         c_same_dim = 1)
```

ρ	$c_{\min}$	$c_{\text{cond indep}}$	$c_{\text{same dim}}$
0.85	0.445	0.722	1
0.50	0.000	0.250	1

```
# cond: "base" (only q5 misfits) | "sweep" (both, cor = c) | "noise" (both, independent)
simdat <- function(n, cond, rho, t, seed) {
  set.seed(seed)
  if (cond == "noise") {
    th <- rnorm(n, 0, PH_SD)
    d <- sim_partial_score(PH_THR, th)
    for (it in c(ITEM_A, ITEM_B)) {
      fl <- runif(n) < NOISE_P
    }
  }
}
```

```

    if (any(fl)) d[fl, it] <- sample(0:MAXCAT, sum(fl), replace = TRUE)
  }
} else if (cond == "base") {
  S <- PH_SD^2 * matrix(c(1, rho, rho, 1), 2, 2)
  th <- MASS::mvrnorm(n, c(0, 0), S)
  d <- sim_partial_score(PH_THR, th[, 1])
  d[, ITEM_A] <- sim_partial_score(PH_THR[ITEM_A], th[, 2])[, 1]
} else {
  cc <- c_of(rho, t)
  S <- PH_SD^2 * matrix(c(1, rho, rho,
                          rho, 1, cc,
                          rho, cc, 1), 3, 3)
  th <- MASS::mvrnorm(n, c(0, 0, 0), S)
  d <- sim_partial_score(PH_THR, th[, 1])
  d[, ITEM_A] <- sim_partial_score(PH_THR[ITEM_A], th[, 2])[, 1]
  d[, ITEM_B] <- sim_partial_score(PH_THR[ITEM_B], th[, 3])[, 1]
}
d <- as.data.frame(d); names(d) <- paste0("q", 1:9); d
}

one <- function(g) {
  out <- try({
    # `t` is NA for the baseline and `rho`/`t` are NA for the noise condition;
    # substitute 0 so the seed arithmetic stays finite.
    rho_s <- if (is.na(g$rho)) 0 else g$rho
    t_s <- if (is.na(g$t)) 0 else g$t
    d <- simdat(g$n, g$cond, g$rho, g$t,
               seed = 4e8 + 1e6 * match(g$cond, c("base", "sweep", "noise")) +
               1e5 * round(rho_s * 100) + 1e4 * round(t_s * 100) +
               1e3 * g$n + g$rep)
    x <- RMitemInfit(d, output = "dataframe")
    data.frame(infit_A = x$Infit_MSQ[ITEM_A], infit_B = x$Infit_MSQ[ITEM_B])
  }, silent = TRUE)
  if (inherits(out, "try-error")) NULL else out
}

grid <- rbind(
  expand.grid(rep = 1:R_REPS, n = NS, rho = RHOS, t = NA_real_,
             cond = "base", stringsAsFactors = FALSE),
  expand.grid(rep = 1:R_REPS, n = NS, rho = RHOS, t = TS,
             cond = "sweep", stringsAsFactors = FALSE),
  expand.grid(rep = 1:R_REPS, n = NS, rho = NA_real_, t = NA_real_,
             cond = "noise", stringsAsFactors = FALSE)
)[, c("cond", "rho", "t", "n", "rep")]

if (file.exists(RES)) {
  res <- readRDS(RES)
} else {
  t0 <- Sys.time()
  out <- mclapply(seq_len(nrow(grid)), function(i) one(grid[i, , drop = FALSE])),
                mc.cores = NCORES)
  res <- list(grid = grid, out = out,
             elapsed_min = as.numeric(Sys.time() - t0, "mins"))
  saveRDS(res, RES)
}

D <- cbind(res$grid,
           do.call(rbind, lapply(res$out, function(x)
             if (is.null(x)) data.frame(infit_A = NA, infit_B = NA) else x)))
cat(nrow(D), "datasets |", round(res$elapsed_min, 1), "min | failed:",
    sum(is.na(D$infit_A)), "\n")

```

Attenuation of the on-target item (q5)

q5 misfits in every condition. The baseline is q5 misfitting **alone**; every other row adds a second misfitting item (q8) and differs only in how correlated the two misfit sources are.

```
base <- D |>
  filter(cond == "base") |>
  group_by(rho, n) |>
  summarise(baseline = mean(infit_A, na.rm = TRUE), .groups = "drop")

D |>
  filter(cond == "sweep") |>
  group_by(rho, t, n) |>
  summarise(q5 = mean(infit_A, na.rm = TRUE),
            q8 = mean(infit_B, na.rm = TRUE), .groups = "drop") |>
  left_join(base, by = c("rho", "n")) |>
  mutate(cor_sources = round(c_of(rho, t), 3),
         attenuation = round(q5 - baseline, 3),
         across(c(q5, q8, baseline), \(x) round(x, 3))) |>
  select(rho, n, t, cor_sources, baseline, q5, q8, attenuation) |>
  arrange(rho, n, t)
```

rho	n	t	cor_sources	baseline	q5	q8	attenuation
0.50	250	0.00	0.445	1.809	1.678	1.589	-0.131
0.50	250	0.25	0.584	1.809	1.629	1.562	-0.180
0.50	250	0.50	0.722	1.809	1.613	1.519	-0.195
0.50	250	0.75	0.861	1.809	1.587	1.465	-0.222
0.50	250	1.00	1.000	1.809	1.541	1.429	-0.268
0.50	1000	0.00	0.445	1.810	1.675	1.596	-0.135
0.50	1000	0.25	0.584	1.810	1.641	1.558	-0.169
0.50	1000	0.50	0.722	1.810	1.607	1.515	-0.203
0.50	1000	0.75	0.861	1.810	1.576	1.472	-0.235
0.50	1000	1.00	1.000	1.810	1.537	1.442	-0.273
0.85	250	0.00	0.445	1.285	1.289	1.267	0.004
0.85	250	0.25	0.584	1.285	1.268	1.245	-0.017
0.85	250	0.50	0.722	1.285	1.240	1.220	-0.045
0.85	250	0.75	0.861	1.285	1.219	1.199	-0.066
0.85	250	1.00	1.000	1.285	1.193	1.166	-0.092
0.85	1000	0.00	0.445	1.277	1.284	1.281	0.007
0.85	1000	0.25	0.584	1.277	1.261	1.249	-0.016
0.85	1000	0.50	0.722	1.277	1.242	1.213	-0.035
0.85	1000	0.75	0.861	1.277	1.222	1.196	-0.056
0.85	1000	1.00	1.000	1.277	1.200	1.164	-0.077

The $c = 0$ anchor, from independent random responding:

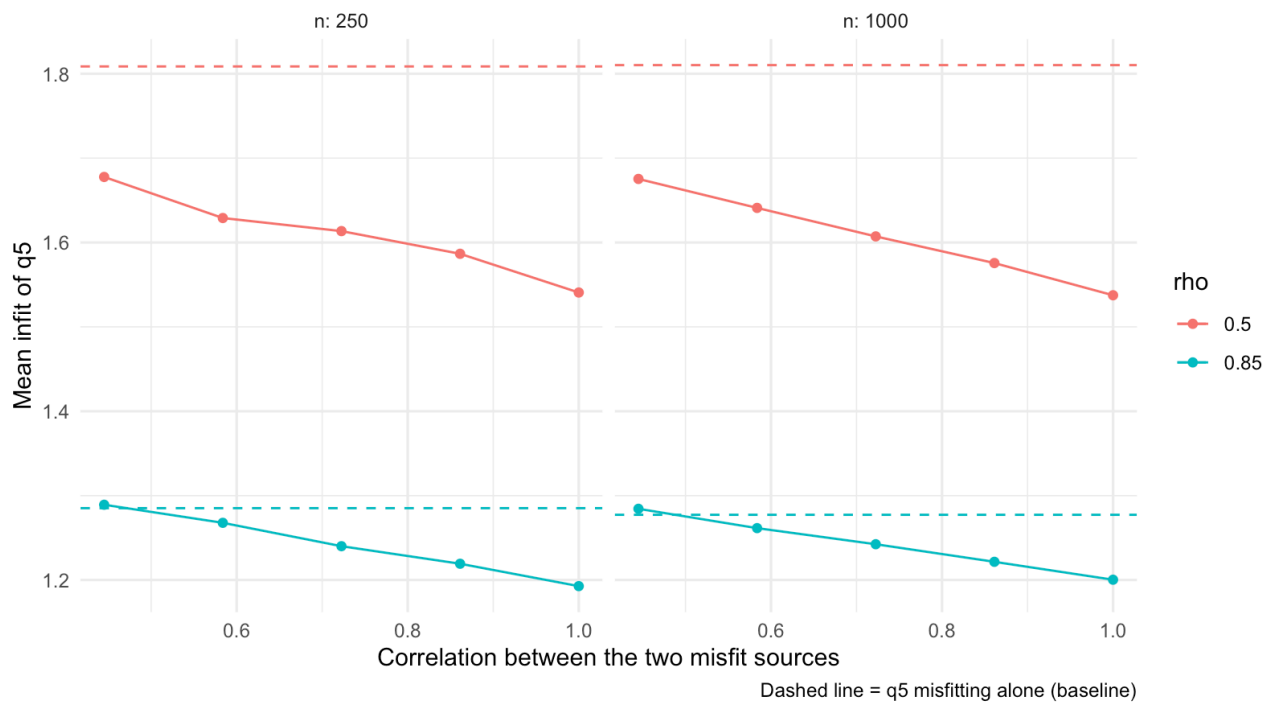
```
D |>
  filter(cond == "noise") |>
```

```
group_by(n) |>
summarise(q5 = round(mean(infit_A, na.rm = TRUE), 3),
          q8 = round(mean(infit_B, na.rm = TRUE), 3), .groups = "drop") |>
mutate(note = "both items misfit independently (cor = 0)")
```

n	q5	q8	note
250	1.194	1.328	both items misfit independently (cor = 0)
1000	1.197	1.338	both items misfit independently (cor = 0)

```
plotd <- D |>
filter(cond == "sweep") |>
group_by(rho, t, n) |>
summarise(q5 = mean(infit_A, na.rm = TRUE), .groups = "drop") |>
mutate(cor_sources = c_of(rho, t))

ggplot(plotd, aes(cor_sources, q5, colour = factor(rho))) +
  geom_line() + geom_point() +
  geom_hline(data = base, aes(yintercept = baseline, colour = factor(rho)),
            linetype = 2) +
  facet_wrap(~n, labeller = label_both) +
  labs(x = "Correlation between the two misfit sources",
       y = "Mean infit of q5",
       colour = "rho",
       caption = "Dashed line = q5 misfitting alone (baseline)") +
  theme_minimal(base_size = 11)
```



Detection follow-up

Does the attenuation above translate into a detection difference? This section adds bootstrap cutoffs to the same conditions.

Sample sizes are $n = 150$ and 250 , not 1000 . At $n = 1000$ detection of a $\rho = 0.85$ misfitting item is already ~92 percent, so there is no room for an 0.08 shift in infit to show; the informative range is where detection sits around 25-50 percent. The attenuation itself was shown above to be n -invariant, so nothing is lost by measuring its consequences at the sample sizes where they are visible.

$B = 2000$ (converged), 120 replications, HDIC $w = .999$ and the FWER p-value.

```

DET_NS <- c(150, 250)
DET_B <- 2000
DET_REPS <- 120
ALPHA <- 0.05
DET_RES <- "infit_misfit_source_correlation_detection.rds"
FITTING <- setdiff(1:9, c(ITEM_A, ITEM_B))

```

```

library(ggdist)

bounds_at <- function(results, width, items) {
  vapply(items, function(it) {
    v <- results$InfitMSQ[results$Item == it]
    as.numeric(ggdist::hdci(v, .width = width))[1:2]
  }, numeric(2))
}

one_det <- function(g) {
  out <- try({
    rho_s <- if (is.na(g$rho)) 0 else g$rho
    t_s <- if (is.na(g$t)) 0 else g$t
    d <- simdat(g$n, g$cond, g$rho, g$t,
      seed = 6e8 + 1e6 * match(g$cond, c("base", "sweep", "noise")) +
        1e5 * round(rho_s * 100) + 1e4 * round(t_s * 100) +
        1e3 * g$n + g$rep)
    items <- names(d)
    obs <- RMitemInfit(d, output = "dataframe")
    msq <- obs$Infit_MSQ[match(items, obs$Item)]
    co <- RMitemInfitCutoff(d, iterations = DET_B, parallel = FALSE,
      seed = g$rep * 100 + 7, verbose = FALSE)
    b <- bounds_at(co$results, 0.999, items)
    hdci <- as.integer(msq > b[2, ] | msq < b[1, ])
    fw <- suppressWarnings(
      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
        alpha = ALPHA, output = "dataframe"))
    fwer <- as.integer(fw$padj_infit[match(items, fw$Item)] < ALPHA)
    data.frame(item = seq_along(items), hdci = hdci, fwer = fwer)
  }, silent = TRUE)
  if (inherits(out, "try-error")) NULL else out
}

det_grid <- rbind(
  expand.grid(rep = 1:DET_REPS, n = DET_NS, rho = RHOS, t = NA_real_,
    cond = "base", stringsAsFactors = FALSE),
  expand.grid(rep = 1:DET_REPS, n = DET_NS, rho = RHOS, t = TS,
    cond = "sweep", stringsAsFactors = FALSE),
  expand.grid(rep = 1:DET_REPS, n = DET_NS, rho = NA_real_, t = NA_real_,
    cond = "noise", stringsAsFactors = FALSE)
)[, c("cond", "rho", "t", "n", "rep")]
det_grid <- det_grid[order(det_grid$n), ]
rownames(det_grid) <- NULL
cat("detection cells:", nrow(det_grid), "\n")

```

detection cells: 3120

```

run_resumable <- function(res_file, grid, keys, fn, ncores = NCORES,
  chunk_size = 100) {
  key_of <- function(g) if (nrow(g) == 0L) character(0) else
    do.call(paste, c(unname(as.list(g[keys])), sep = "|"))
  acc <- if (file.exists(res_file)) readRDS(res_file) else
    list(res = list(), grid = grid[0, , drop = FALSE], elapsed_min = 0)

```

```

todo <- which(!(key_of(grid) %in% key_of(acc$grid)))
if (length(todo) == 0L) { message("cached: nothing to run"); return(acc) }
message("running ", length(todo), " new cells")
for (s in seq(1, length(todo), by = chunk_size)) {
  idx <- todo[s:min(s + chunk_size - 1L, length(todo))]
  t0 <- Sys.time()
  o <- mclapply(idx, function(i) fn(grid[i, , drop = FALSE]), mc.cores = ncores)
  acc$res <- c(acc$res, o)
  acc$grid <- rbind(acc$grid, grid[idx, , drop = FALSE])
  acc$elapsed_min <- acc$elapsed_min + as.numeric(Sys.time() - t0, "mins")
  saveRDS(acc, res_file)
  message(" checkpoint ", nrow(acc$grid), " / ", nrow(grid))
}
acc
}

ds <- run_resumable(DET_RES, det_grid, c("cond", "rho", "t", "n", "rep"), one_det)

DET <- do.call(rbind, lapply(seq_len(nrow(ds$grid)), function(i) {
  r <- ds$res[[i]]
  if (is.null(r)) return(NULL)
  cbind(ds$grid[i, c("cond", "rho", "t", "n", "rep")], r)
}))
cat(nrow(ds$grid), "cells |", round(ds$elapsed_min, 1), "min | failed:",
    sum(sapply(ds$res, is.null)), "\n")

```

```
3120 cells | 244.9 min | failed: 0
```

Detection of q5, across the correlation sweep

q5 misfits in every condition, so this is a clean within-item comparison. The baseline row is q5 misfitting alone.

```

det_base <- DET |>
  filter(cond == "base", item == ITEM_A) |>
  group_by(rho, n) |>
  summarise(base_hdci = 100 * mean(hdci), base_fwer = 100 * mean(fwer),
    .groups = "drop")

DET |>
  filter(cond == "sweep", item == ITEM_A) |>
  group_by(rho, t, n) |>
  summarise(hdci = 100 * mean(hdci), fwer = 100 * mean(fwer), .groups = "drop") |>
  left_join(det_base, by = c("rho", "n")) |>
  mutate(cor_src = round(c_of(rho, t), 3),
    d_hdci = round(hdci - base_hdci, 1),
    d_fwer = round(fwer - base_fwer, 1),
    across(c(hdci, fwer, base_hdci, base_fwer), \(x) round(x, 1))) |>
  select(rho, n, cor_src, base_hdci, hdci, d_hdci, base_fwer, fwer, d_fwer) |>
  arrange(rho, n, cor_src)

```

rho	n	cor_src	base_hdci	hdci	d_hdci	base_fwer	fwer	d_fwer
0.50	150	0.445	100.0	99.2	-0.8	100.0	100.0	0.0
0.50	150	0.584	100.0	97.5	-2.5	100.0	99.2	-0.8
0.50	150	0.722	100.0	95.0	-5.0	100.0	98.3	-1.7
0.50	150	0.861	100.0	90.8	-9.2	100.0	97.5	-2.5
0.50	150	1.000	100.0	92.5	-7.5	100.0	97.5	-2.5
0.50	250	0.445	100.0	100.0	0.0	100.0	100.0	0.0
0.50	250	0.584	100.0	100.0	0.0	100.0	100.0	0.0

rho	n	cor_src	base_hdci	hdci	d_hdci	base_fwer	fwer	d_fwer
0.50	250	0.722	100.0	100.0	0.0	100.0	100.0	0.0
0.50	250	0.861	100.0	100.0	0.0	100.0	100.0	0.0
0.50	250	1.000	100.0	98.3	-1.7	100.0	100.0	0.0
0.85	150	0.445	26.7	27.5	0.8	45.8	47.5	1.7
0.85	150	0.584	26.7	22.5	-4.2	45.8	40.0	-5.8
0.85	150	0.722	26.7	15.8	-10.8	45.8	31.7	-14.2
0.85	150	0.861	26.7	10.8	-15.8	45.8	22.5	-23.3
0.85	150	1.000	26.7	9.2	-17.5	45.8	25.0	-20.8
0.85	250	0.445	51.7	49.2	-2.5	69.2	70.8	1.7
0.85	250	0.584	51.7	40.0	-11.7	69.2	60.8	-8.3
0.85	250	0.722	51.7	40.8	-10.8	69.2	60.0	-9.2
0.85	250	0.861	51.7	30.0	-21.7	69.2	48.3	-20.8
0.85	250	1.000	51.7	23.3	-28.3	69.2	33.3	-35.8

False positives on the seven fitting items

```
DET |>
  filter(cond == "sweep", item %in% FITTING) |>
  group_by(rho, t, n) |>
  summarise(hdci = round(100 * mean(hdci), 1),
            fwer = round(100 * mean(fwer), 1), .groups = "drop") |>
  mutate(cor_src = round(c_of(rho, t), 3)) |>
  select(rho, n, cor_src, hdci, fwer) |>
  arrange(rho, n, cor_src)
```

rho	n	cor_src	hdci	fwer
0.50	150	0.445	7.5	11.8
0.50	150	0.584	5.6	9.2
0.50	150	0.722	4.6	8.3
0.50	150	0.861	3.6	5.7
0.50	150	1.000	2.9	4.2
0.50	250	0.445	19.4	33.7
0.50	250	0.584	15.7	28.0
0.50	250	0.722	10.1	19.4
0.50	250	0.861	7.5	16.0
0.50	250	1.000	6.3	13.6
0.85	150	0.445	1.0	0.8
0.85	150	0.584	0.4	0.6
0.85	150	0.722	0.4	0.5
0.85	150	0.861	0.4	0.6
0.85	150	1.000	0.2	0.2
0.85	250	0.445	0.7	1.4
0.85	250	0.584	0.8	1.9

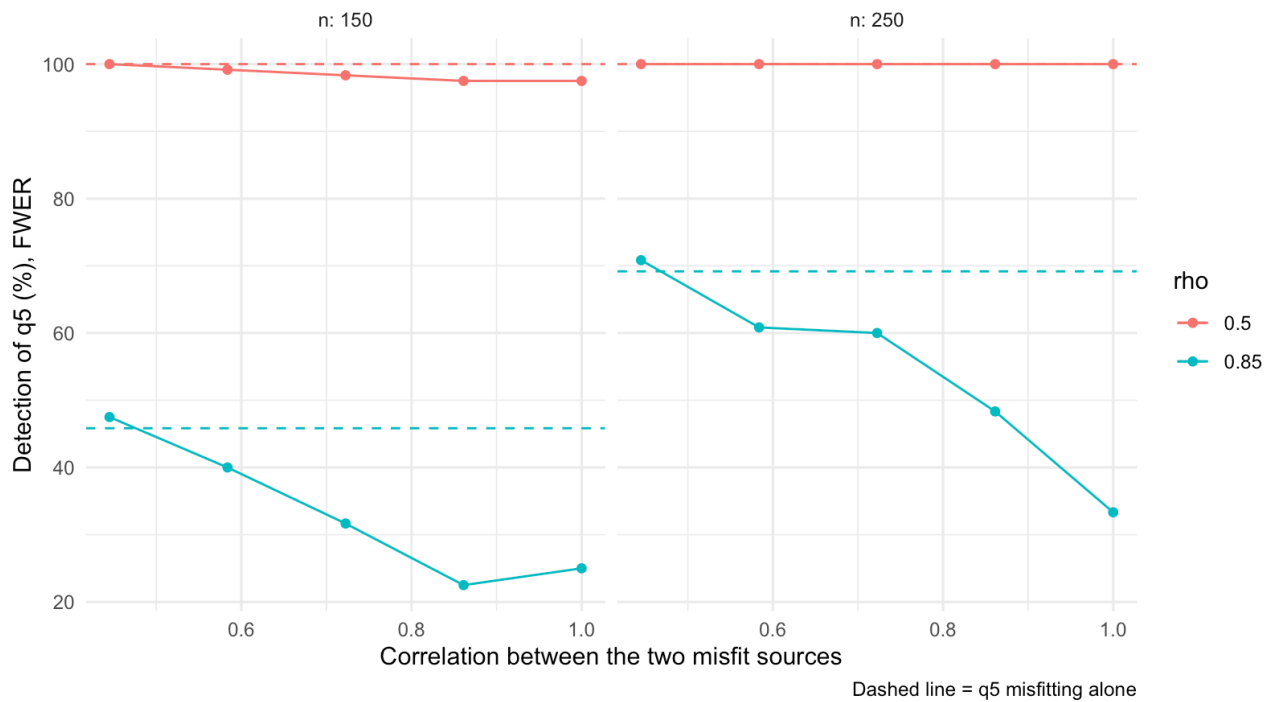
rho	n	cor_src	hdci	fwer
0.85	250	0.722	0.4	1.0
0.85	250	0.861	0.5	1.0
0.85	250	1.000	0.5	0.5

The c = 0 anchor

```
DET |>
  filter(cond == "noise") |>
  mutate(grp = case_when(item == ITEM_A ~ "q5 (misfit)",
                        item == ITEM_B ~ "q8 (misfit)",
                        TRUE ~ "fitting items")) |>
  group_by(n, grp) |>
  summarise(hdci = round(100 * mean(hdci), 1),
            fwer = round(100 * mean(fwer), 1), .groups = "drop")
```

n	grp	hdci	fwer
150	fitting items	0.5	1.0
150	q5 (misfit)	10.0	24.2
150	q8 (misfit)	36.7	56.7
250	fitting items	1.0	2.1
250	q5 (misfit)	21.7	44.2
250	q8 (misfit)	59.2	76.7

```
DET |>
  filter(cond == "sweep", item == ITEM_A) |>
  group_by(rho, t, n) |>
  summarise(fwer = 100 * mean(fwer), .groups = "drop") |>
  mutate(cor_src = c_of(rho, t)) |>
  ggplot(aes(cor_src, fwer, colour = factor(rho))) +
  geom_line() + geom_point() +
  geom_hline(data = det_base, aes(yintercept = base_fwer, colour = factor(rho)),
            linetype = 2) +
  facet_wrap(~n, labeller = label_both) +
  labs(x = "Correlation between the two misfit sources",
       y = "Detection of q5 (%), FWER",
       colour = "rho",
       caption = "Dashed line = q5 misfitting alone") +
  theme_minimal(base_size = 11)
```



Notes for the write-up

Two points to carry into the paper:

1. **Attenuation is graded by the correlation between misfit sources**, not by whether they are nominally “the same dimension”. Conditional infit conditions on the total score, so a second misfitting item makes the first look less deviant only to the extent that its deviation is correlated with the first item’s — which is why independent random responding produces no attenuation at all.
2. **The admissible range of that correlation is bounded below by $2 \cdot \rho^2 - 1$** . At realistic misfit levels nearly the whole range is compressed toward 1, so same-dimension and separate-dimension designs are close to indistinguishable. This is a property of the geometry, not of the fit statistic, and it explains why earlier informal comparisons found little difference.

Detection consequences are not measured here — that requires bootstrap cutoffs and is a separate run.

References

Johansson, M. (2025). Detecting Item Misfit in Rasch Models. *Educational Methods & Psychometrics*, 3(18).